



UNIVERSIDADE FEDERAL DO AMAPÁ
CURSO DE CIÊNCIA DA COMPUTAÇÃO

DANIELA MARQUES HABER SEPEDA

Atividade de Compiladores
Docente: Anderson dos Santos Guerra

Tarefa: Construa a árvore de derivação completa para cada uma das declarações MicroJava abaixo, de acordo com a gramática formal da linguagem (MicroJava Quick Reference).

```
sum = a + max;  
total = arr[0] + arr[1];  
values = new int[size];  
arr = new int[10];  
letter = 'A';  
class Node { int data; int[] next; }  
if (x > 0) x = x - 1; else x = x + 1;  
while (i < size) i = i + 1;
```

Resposta:

As árvores de derivação a seguir foram construídas a partir da gramática EBNF do *MicroJava Quick Reference*. Em cada árvore, os símbolos **não-terminais** aparecem dentro de caixas arredondadas; os **terminais literais** (palavras reservadas, operadores e pontuação) aparecem em negrito; e as **classes terminais léxicas** (ident, number, charConst) aparecem sublinhadas, com o lexema correspondente indicado logo abaixo, entre parênteses. Antes de cada árvore é apresentada também a **derivação mais à esquerda** completa, passo a passo, até a forma sentencial composta apenas por símbolos terminais. Vale notar que, em MicroJava, int é um nome de tipo pré-declarado, reconhecido pelo analisador léxico como ident.

Gramática de referência (produções utilizadas):

```
Statement = Designator ("=" Expr | ActPars) ";"  
           | "if" "(" Condition ")" Statement ["else" Statement]  
           | "while" "(" Condition ")" Statement | ... .  
ClassDecl  = "class" ident "{" {VarDecl} "}".  
VarDecl    = Type ident {"", " ident} ";".           Type = ident ["[" "]""]".  
Condition  = Expr Relop Expr.                       Relop = "==" | "!=" | ">" | ">=" | "<" | "<=".  
Expr       = ["-"] Term {Addop Term}.                 Term = Factor {Mulop Factor}.  
Factor     = Designator [ActPars] | number | charConst | "new" ident ["[" Expr "]""] | "(" Expr ")".  
Designator = ident {"." ident | "[" Expr "]""]. Addop = "+" | "-". Mulop = "*" | "/" | "%".
```

Declaração 1: $sum = a + max;$

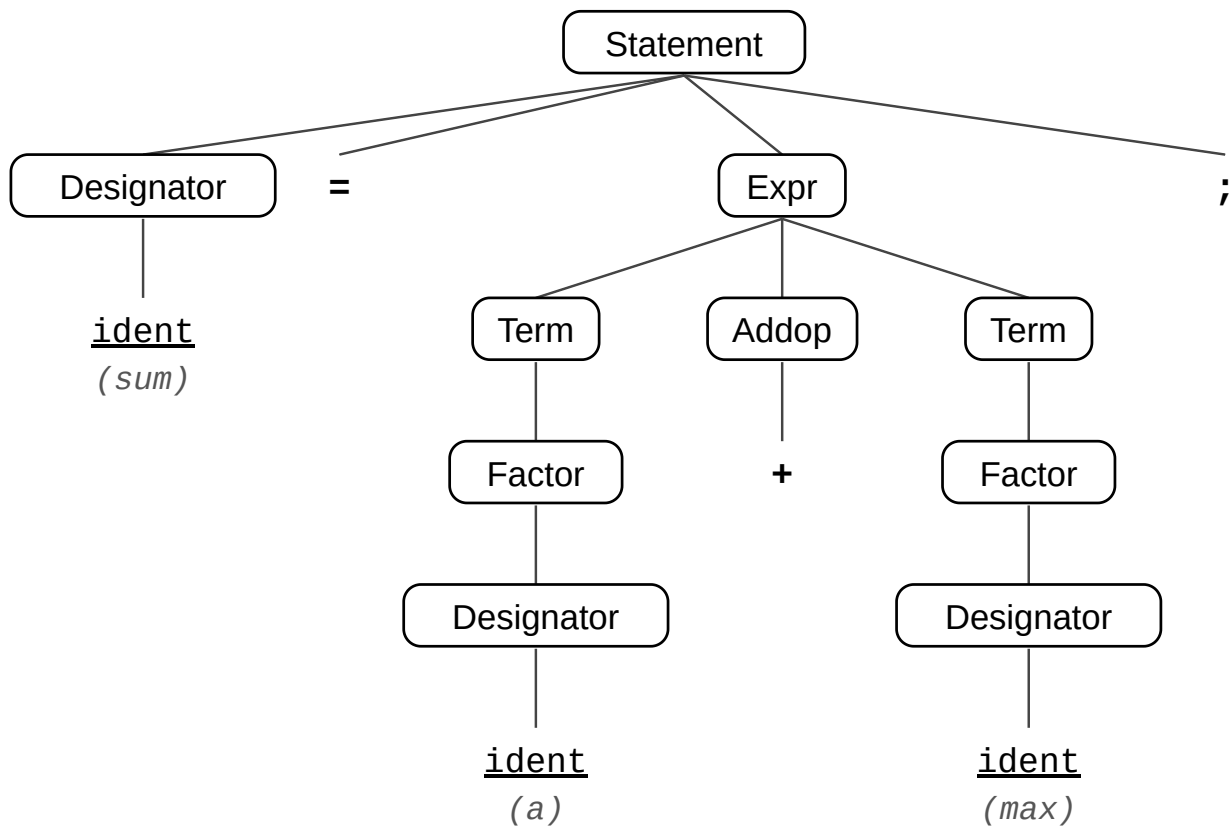
a) Derivação mais à esquerda:

Statement

⇒ Designator "=" Expr ";"
⇒ ident "=" Expr ";"
⇒ ident "=" Term Addop Term ";"
⇒ ident "=" Factor Addop Term ";"
⇒ ident "=" Designator Addop Term ";"
⇒ ident "=" ident Addop Term ";"
⇒ ident "=" ident "+" Term ";"
⇒ ident "=" ident "+" Factor ";"
⇒ ident "=" ident "+" Designator ";"
⇒ ident "=" ident "+" ident ";"

Sentença derivada: $sum = a + max;$

b) Árvore de derivação:



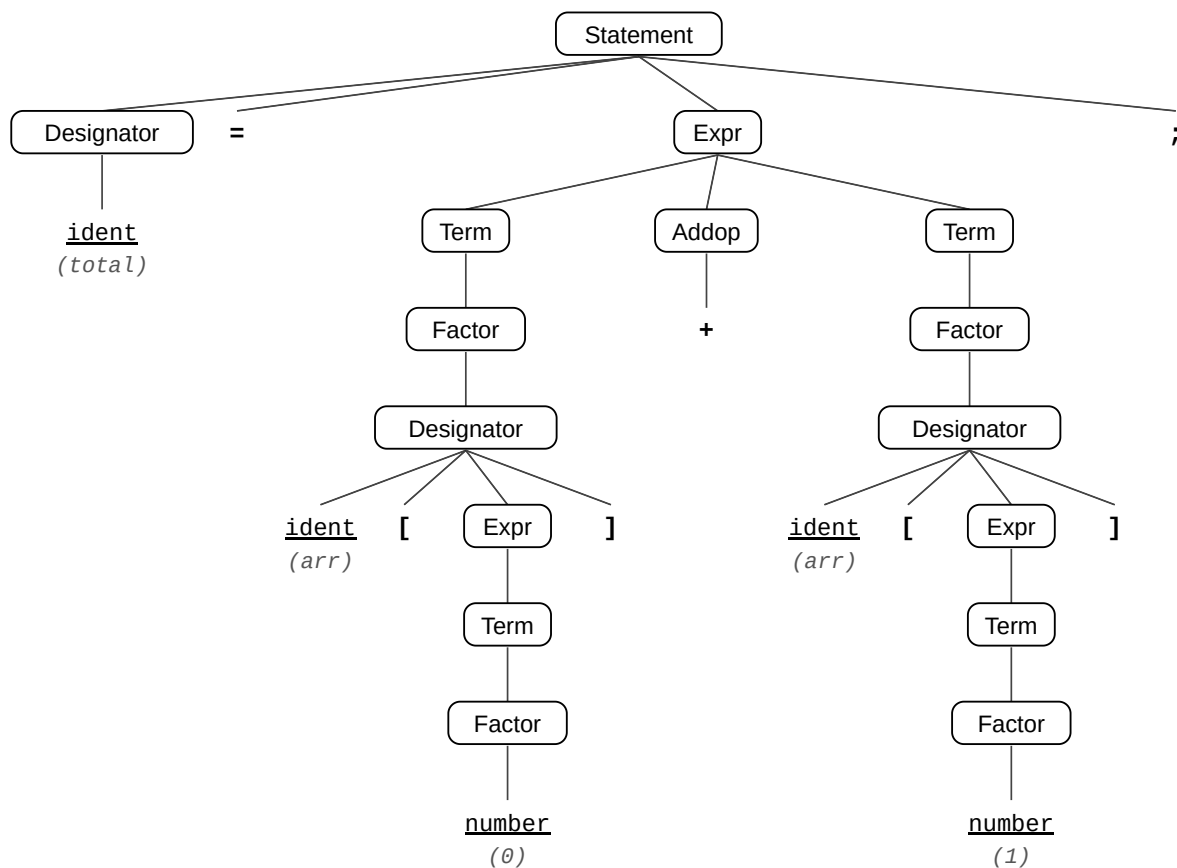
Declaração 2: total = arr[0] + arr[1];

a) Derivação mais à esquerda:

Statement	⇒ ident "=" ident "[" number "]" "+" Term ";"
⇒ Designator "=" Expr ";"	⇒ ident "=" ident "[" number "]" "+" Factor ";"
⇒ ident "=" Expr ";"	⇒ ident "=" ident "[" number "]" "+" Designator ";"
⇒ ident "=" Term Addop Term ";"	⇒ ident "=" ident "[" number "]" "+" ident "[" Expr "]" ";"
⇒ ident "=" Factor Addop Term ";"	⇒ ident "=" ident "[" number "]" "+" ident "[" Term "]" ";"
⇒ ident "=" Designator Addop Term ";"	⇒ ident "=" ident "[" number "]" "+" ident "[" Factor "]" ";"
⇒ ident "=" ident "[" Expr "]" Addop Term ";"	⇒ ident "=" ident "[" number "]" "+" ident "[" number "]" ";"
⇒ ident "=" ident "[" Term "]" Addop Term ";"	
⇒ ident "=" ident "[" Factor "]" Addop Term ";"	
⇒ ident "=" ident "[" number "]" Addop Term ";"	

Sentença derivada: *total = arr[0] + arr[1];*

b) Árvore de derivação:



Declaração 3: values = new int[size];

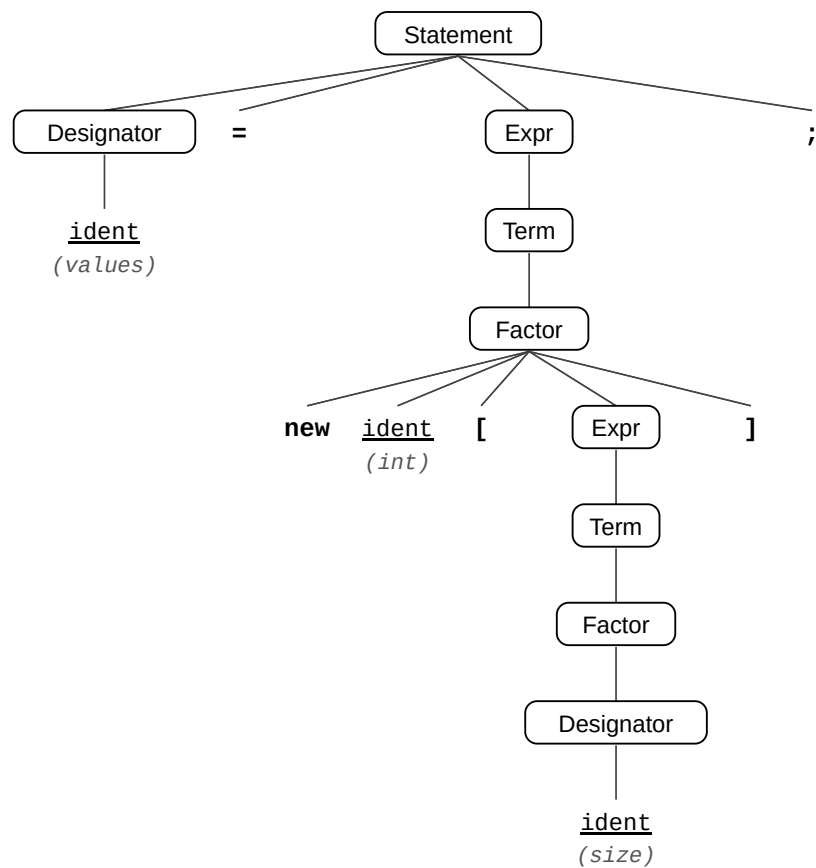
a) Derivação mais à esquerda:

Statement

⇒ Designator "=" Expr ";"
⇒ ident "=" Expr ";"
⇒ ident "=" Term ";"
⇒ ident "=" Factor ";"
⇒ ident "=" "new" ident "[" Expr "]" ";"
⇒ ident "=" "new" ident "[" Term "]" ";"
⇒ ident "=" "new" ident "[" Factor "]" ";"
⇒ ident "=" "new" ident "[" Designator "]" ";"
⇒ ident "=" "new" ident "[" ident "]" ";"

Sentença derivada: *values = new int[size];*

b) Árvore de derivação:



Declaração 4: `arr = new int[10];`

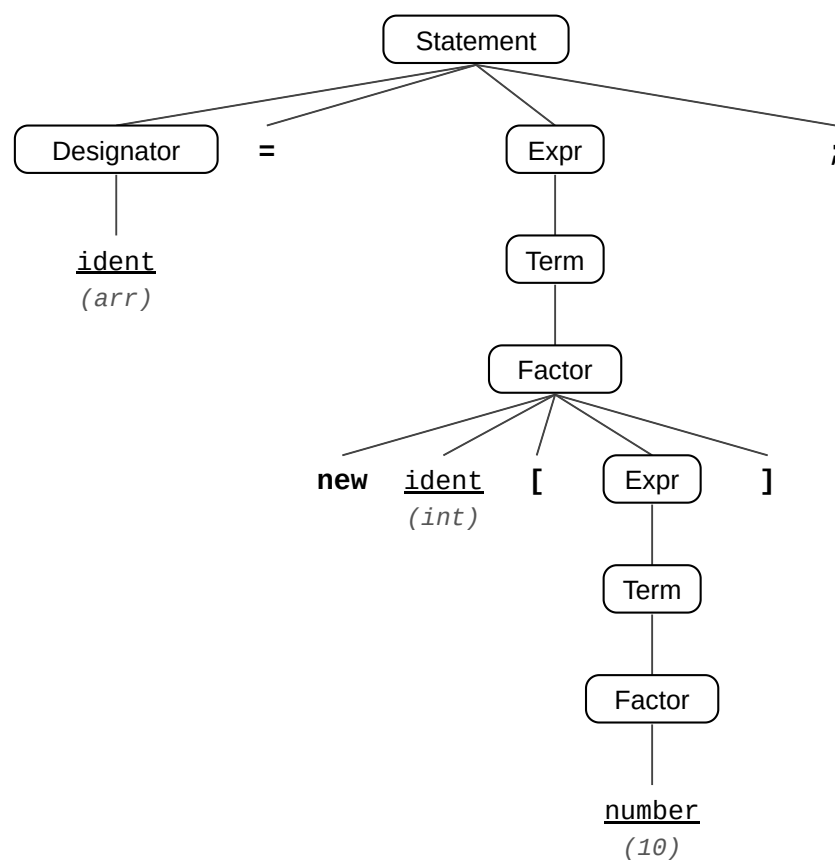
a) Derivação mais à esquerda:

Statement

⇒ Designator "=" Expr ";"
⇒ ident "=" Expr ";"
⇒ ident "=" Term ";"
⇒ ident "=" Factor ";"
⇒ ident "=" "new" ident "[" Expr "]" ";"
⇒ ident "=" "new" ident "[" Term "]" ";"
⇒ ident "=" "new" ident "[" Factor "]" ";"
⇒ ident "=" "new" ident "[" number "]" ";"

Sentença derivada: `arr = new int[10];`

b) Árvore de derivação:



Declaração 5: letter = 'A';

a) Derivação mais à esquerda:

Statement

⇒ Designator "=" Expr ";"

⇒ ident "=" Expr ";"

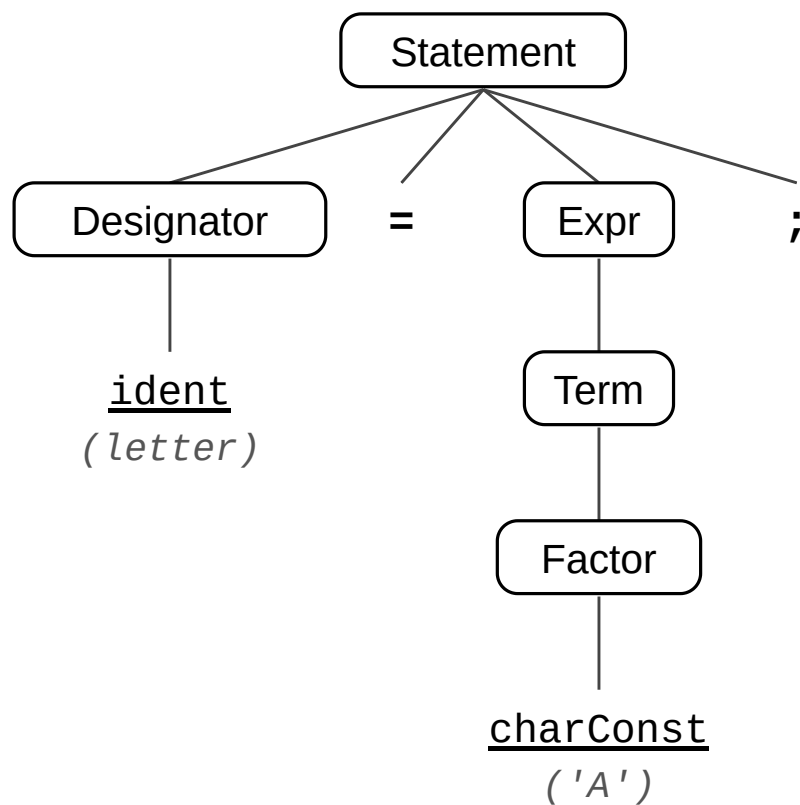
⇒ ident "=" Term ";"

⇒ ident "=" Factor ";"

⇒ ident "=" charConst ";"

Sentença derivada: *letter* = 'A';

b) Árvore de derivação:



Declaração 6: class Node { int data; int[] next; }

a) Derivação mais à esquerda:

ClassDecl

⇒ "class" ident "{" VarDecl VarDecl "}"

⇒ "class" ident "{" Type ident ";" VarDecl "}"

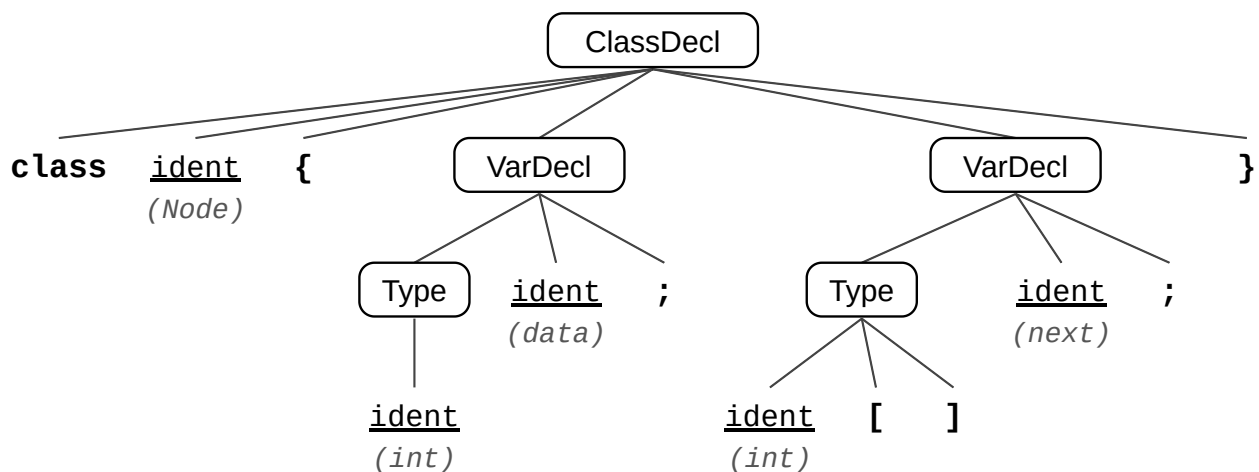
⇒ "class" ident "{" ident ident ";" VarDecl "}"

⇒ "class" ident "{" ident ident ";" Type ident ";" "}"

⇒ "class" ident "{" ident ident ";" ident "[" "]" ident ";" "}"

Sentença derivada: *class Node { int data; int[] next; }*

b) Árvore de derivação:



Declaração 7: `if (x > 0) x = x - 1; else x = x + 1;`

a) Derivação mais à esquerda:

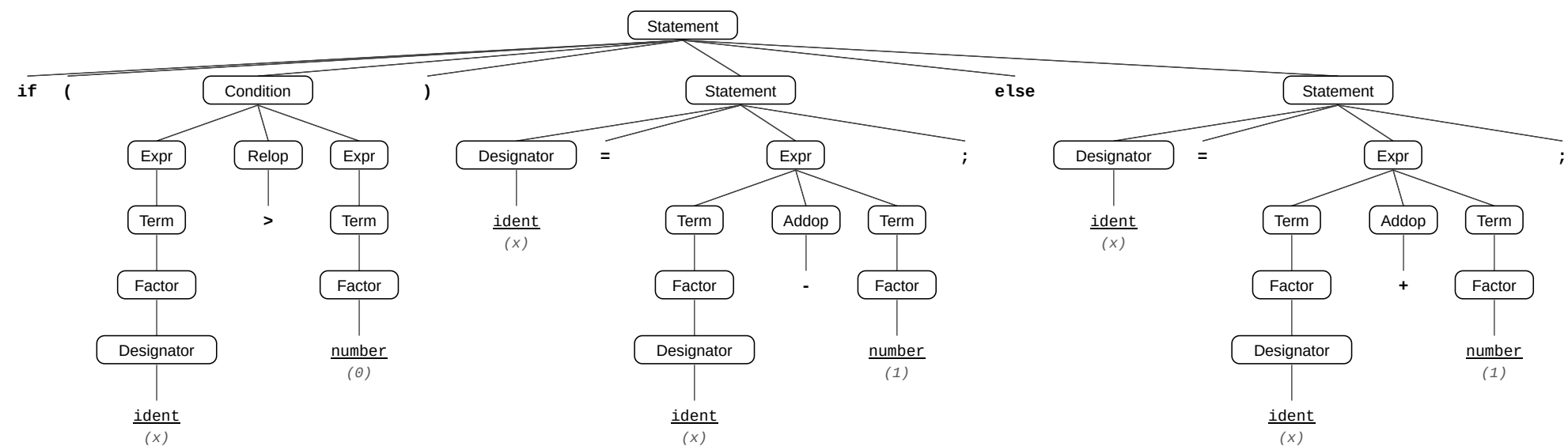
Statement

```
⇒ "if" "(" Condition ")" Statement "else" Statement
⇒ "if" "(" Expr Relop Expr ")" Statement "else" Statement
⇒ "if" "(" Term Relop Expr ")" Statement "else" Statement
⇒ "if" "(" Factor Relop Expr ")" Statement "else" Statement
⇒ "if" "(" Designator Relop Expr ")" Statement "else" Statement
⇒ "if" "(" ident Relop Expr ")" Statement "else" Statement
⇒ "if" "(" ident ">" Expr ")" Statement "else" Statement
⇒ "if" "(" ident ">" Term ")" Statement "else" Statement
⇒ "if" "(" ident ">" Factor ")" Statement "else" Statement
⇒ "if" "(" ident ">" number ")" Statement "else" Statement
⇒ "if" "(" ident ">" number ")" Designator "=" Expr ";" "else"
    Statement
⇒ "if" "(" ident ">" number ")" ident "=" Expr ";" "else" Statement
⇒ "if" "(" ident ">" number ")" ident "=" Term Addop Term ";" "else"
    Statement
⇒ "if" "(" ident ">" number ")" ident "=" Factor Addop Term ";"
    "else" Statement
⇒ "if" "(" ident ">" number ")" ident "=" Designator Addop Term ";"
    "else" Statement
⇒ "if" "(" ident ">" number ")" ident "=" ident Addop Term ";" "else"
    Statement
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" Term ";" "else"
    Statement
```

```
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" Factor ";" "else"
    Statement
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    Statement
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    Designator "=" Expr ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" Expr ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" Term Addop Term ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" Factor Addop Term ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" Designator Addop Term ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" ident Addop Term ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" ident "+" Term ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" ident "+" Factor ";"
⇒ "if" "(" ident ">" number ")" ident "=" ident "-" number ";" "else"
    ident "=" ident "+" number ";"
```

Sentença derivada: *if* ($x > 0$) $x = x - 1$; *else* $x = x + 1$;

b) Árvore de derivação:



Declaração 8: while (i < size) i = i + 1;

a) Derivação mais à esquerda:

Statement

⇒ "while" "(" Condition ")" Statement
⇒ "while" "(" Expr Relop Expr ")" Statement
⇒ "while" "(" Term Relop Expr ")" Statement
⇒ "while" "(" Factor Relop Expr ")"
Statement
⇒ "while" "(" Designator Relop Expr ")"
Statement
⇒ "while" "(" ident Relop Expr ")"
Statement
⇒ "while" "(" ident "<" Expr ")" Statement
⇒ "while" "(" ident "<" Term ")" Statement
⇒ "while" "(" ident "<" Factor ")"
Statement
⇒ "while" "(" ident "<" Designator ")"
Statement
⇒ "while" "(" ident "<" ident ")" Statement
⇒ "while" "(" ident "<" ident ")"
Designator "=" Expr ";"

⇒ "while" "(" ident "<" ident ")" ident "="
Expr ";"
⇒ "while" "(" ident "<" ident ")" ident "="
Term Addop Term ";"
⇒ "while" "(" ident "<" ident ")" ident "="
Factor Addop Term ";"
⇒ "while" "(" ident "<" ident ")" ident "="
Designator Addop Term ";"
⇒ "while" "(" ident "<" ident ")" ident "="
ident Addop Term ";"
⇒ "while" "(" ident "<" ident ")" ident "="
ident "+" Term ";"
⇒ "while" "(" ident "<" ident ")" ident "="
ident "+" Factor ";"
⇒ "while" "(" ident "<" ident ")" ident "="
ident "+" number ";"

Sentença derivada: while (i < size) i = i + 1;

b) Árvore de derivação:

